



Digital Storemesh Co., Ltd. (Head Office)

131 Innovation Group Building 1, Room No. INC1-217, Moo 9, Khlong Nueng, Khlong Luang, Pathum Thani 12120

Data Engineer Technical Assignment

Position Data Engineer

Direction

1. Complete the data engineer technical assignment (as outlined in the attached document)
2. Please submit the related deliverables to company email at contact@storemesh.com within three days from the day you receive this assignment.



Digital Storemesh Co., Ltd. (Head Office)

131 Innovation Group Building 1, Room No. INC1-217, Moo 9, Khlong Nueng, Khlong Luang, Pathum Thani 12120

Data Engineer Technical Assessment

Background

Welcome! At "ShopData Inc.", we are migrating our legacy order management system into a new analytics data warehouse. Our raw data is currently exposed via SQLite views, but it is known to have historical inconsistencies, missing values, and formatting issues.

Your goal is to build an ETL pipeline that extracts this raw data, cleans it, and loads it into a clean analytical format so our BI team can report on Customer Lifetime Value (CLV).

Prerequisites

- Python 3.12+
- Prefect (3.x) for pipeline orchestration
- pytest (or unittest) for writing unit tests
- SQLite (You will be provided with a shopdata.db file containing the source data)
- pandas or pure SQL for transformations (your choice)
- Git (We will be evaluating your commit history and branching strategy)

Source Data

You have read-only access to the following views in the provided shopdata.db database:

1. vw_raw_customers: Contains customer demographics and contact info.
2. vw_raw_orders: Contains transactional data.



Digital Storemesh Co., Ltd. (Head Office)

131 Innovation Group Building 1, Room No. INC1-217, Moo 9, Khlong Nueng, Khlong Luang, Pathum Thani 12120

3. `vw_exchange_rates`: Contains daily exchange rates to USD.

Tasks

Part 1: Data Exploration & Understanding (SQL)

Before writing any Python code, we want to understand the state of the data. Write a short SQL script (`exploration.sql`) containing queries that uncover anomalies in the `vw_raw_customers` and `vw_raw_orders` views.

- Deliverable: Add a brief section in your `README.md` summarizing at least 3 distinct data quality issues you discovered.

Part 2: Data Cleaning & Pipeline Orchestration (Python + Prefect)

Create a Prefect Flow (`pipeline.py`) that performs the following ETL steps:

Extract:

- Read data from the views in the `shopdata.db` SQLite database.

Transform (Cleaning Rules):

1. Customers:

- Deduplicate records (assume the most recent `signup_date` is the correct, updated record).
- Standardize the phone column to remove all non-numeric characters (e.g., `+1 (555) 123-4567` becomes `15551234567`). Replace missing emails with `"unknown@domain.com"`.

2. Orders:



Digital Storemesh Co., Ltd. (Head Office)

131 Innovation Group Building 1, Room No. INC1-217, Moo 9, Khlong Nueng, Khlong Luang, Pathum Thani 12120

- Filter out any orders with a total_amount less than or equal to zero (these are system errors).
- Convert all order amounts to USD (usd_amount) using the vw_exchange_rates view based on the order_date. If a currency is missing or doesn't have an exchange rate, assume it is already 'USD'.

Load:

- Write the cleaned data to a new SQLite database file named analytics.db as two tables: dim_customers and fct_orders. (If you encounter issues creating a new database, outputting them locally as clean_customers.csv and clean_orders.csv is an acceptable fallback).
- **Requirements:**
 - Use Prefect @task and @flow decorators appropriately.
 - Include proper logging and error handling within your tasks.

Part 3: Unit Testing (Python)

We value robust code. Write unit tests for your data transformation logic to ensure your cleaning rules work as expected.

- **Requirements:**
 - Use a framework like pytest.
 - Test at least two distinct transformation functions (e.g., the phone number standardizer and the currency conversion logic).
 - You should test the logic independently of the database (i.e., pass dummy data or DataFrames into your functions rather than relying on a live DB connection).



Part 4: Analytical Query (SQL)

Using your newly cleaned data (`dim_customers` and `fct_orders`), write a single SQL query (`clv_report.sql`) that calculates the Customer Lifetime Value (CLV). The output should include:

- `customer_id`
- `full_name`
- `total_orders_placed`
- `lifetime_value_usd` (Sum of all valid USD order amounts)
- `customer_cohort` (The month and year they signed up, e.g., '2023-01')

Rank the results by `lifetime_value_usd` in descending order.

Deliverables

Please submit a link to a Git repository (GitHub, GitLab, or Bitbucket) containing:

1. `pipeline.py`: Your Prefect pipeline code.
2. `test_pipeline.py` (or a `tests/` directory): Your unit tests.
3. `exploration.sql` & `clv_report.sql`: Your SQL queries.
4. `requirements.txt`: Python dependencies.
5. `README.md`: Instructions on how to run your flow and tests, and a summary of your data exploration findings.

Evaluation Criteria

- **Data Quality:** Did you catch the duplicates, nulls, and negative values?
- **Code Quality:** Is your Python code modular, readable, and appropriately documented?
- **Testing:** Are your unit tests effective and do they test the business logic in isolation?



Digital Storemesh Co., Ltd. (Head Office)

131 Innovation Group Building 1, Room No. INC1-217, Moo 9, Khlong Nueng, Khlong Luang, Pathum Thani 12120

- **Git Practices:** We will look at your commit history. Are your commit messages descriptive?
Did you commit iteratively rather than a single massive commit at the end?
- **Tool Mastery:** Did you effectively use Prefect tasks, flows, and logging?
- **SQL Proficiency:** Are your SQL queries efficient and accurate?